

Dismiss Alert & Alpine JS

We are going to make it so the alert goes away after a few seconds. Since this is something that is interactive and happens on the client-side, we need to use JavaScript. You can use Vanilla JS if you want by adding it to the `public/js/script.js` file, however I prefer to use Alpine JS for this because it is cleaner and we don't even need to write any JavaScript. We do it all from the Blade file. Alpine is a library that gives us a bunch of attributes and directives that we can use in our HTML and it is often used with Laravel to make things more interactive.

Alpine CDN

There are a few ways to use Alpine JS. We can install it via npm or yarn, but we can also use a CDN. We are going to use the CDN. This makes things much easier. Open the `resources/views/components/layout.blade.php` file and add the following code to the `<head>` tag:

```
<script
  defer
  src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
></script>
```

Alpine Directives

Now open the alert component at `resources/views/components/Alert.blade.php` and add the following code:

```
@props(['type', 'message'])

<div
  x-data="{ show: true }"
  x-init="setTimeout(() => show = false, 5000)"
  x-show="show"
  class="p-4 mb-4 text-sm text-white {{ $type === 'success' ? 'bg-green-500' :
'bg-red-500' }} rounded">
  {{ session($type) }}
</div>
```

We are using the `x-data` directive to create a new object with a `show` property. We are setting it to `true` by default. We are using the `x-init` directive to set a timeout of 5 seconds to set the `show` property to `false`. We are using the `x-show` directive to show the alert if `show` is `true`.

Just to show you what we would need to do if we were using Vanilla JS, we can add the following code to the `public/js/script.js` file:

```
document.querySelectorAll('[id$="-alert"]').forEach(function (alert) {  
  // Get the duration from the data attribute  
  const duration = parseInt(alert.getAttribute('data-duration'), 10);  
  
  // Set a timeout to remove the alert after the specified duration  
  setTimeout(function () {  
    alert.style.opacity = 0;  
    setTimeout(function () {  
      alert.remove();  
    }, 600); // Match this duration with your CSS transition time  
  }, duration);  
});
```

You can see why Alpine JS is so much easier to use.

Now your alert should go away after a few seconds.

Using Alpine JS For The Mobile Menu

We have our hamburger menu working with vanilla JS, but we can use Alpine JS to make it even easier. Open the `resources/views/components/header.blade.php` file and make some changes.

First, we need to add an "open" state. This has to be on a parent of the button and menu. So let's add it to the `<header>` element and set it to false:

```
<header class="bg-blue-900 text-white p-4" x-data="{ open: false }">...</header>
```

Next, let's have the hamburger menu button toggle the open state. We can do this by adding the following code to the button:

```
<button @click="open = !open" class="text-white md:hidden flex items-center">  
  <i class="fa fa-bars text-2xl"></i>  
</button>
```

I removed the id of `hamburger`. We don't need it anymore. We are using the `@click` directive to toggle the `open` property. If it is `true`, it will be set to `false` and vice versa.

Finally, we need to show the menu if `open` is `true`. We can do this by adding the following code to the `<nav>` element:

```
<nav  
  x-show="open"  
  @click.away="open = false"  
  class="md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"  
></nav>
```

I removed the class of `hidden`. We don't need it anymore. We are using the `x-show` directive to show the menu if `open` is `true`. We are using the `@click.away` directive to close the menu if the user clicks away from it.

Now you can delete the Javascript from the `public/js/script.js` file. We don't need it anymore. The mobile menu should still work the same way.